

20. WiFi 文字列テレメトリ bridge

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	5
6 このファイルを読む前に必要な基礎知識	6
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	6
6.2 名前、代入、型、参照	6
6.3 関数、引数、戻り値、スコープ	6
6.4 module、package、import、console_scripts	7
6.5 例外、try/finally、with、資源解放	7
6.6 class、独自クラス、インスタンス、self、継承	7
6.7 先頭アンダースコア、__init__、名前付けの作法	8
6.8 rclpy、rcl、rmw、DDS/RTSPS の層	8
6.9 ROS graph、Node、topic、service、action、parameter	8
6.10 callback、timer、Executor、spin、Callback Group	9
6.11 rqt_graph、ros2 CLI、rosvbag2 をいつ使うか	9
6.12 制御、状態、入力、モデル予測制御 MPC	10
7 関数・クラスを上から順に解説	10
7.1 L19 関数 _safe_float	10

7.2	L32 関数 <code>_parse_key_value_text</code>	11
7.3	L53 関数 <code>parse_text_payload</code>	13
7.4	L66 関数 <code>headwind_component_ms</code>	14
7.5	L79 クラス <code>TelemetryTextBridgeNode</code>	15
7.6	L80 関数 <code>TelemetryTextBridgeNode.__init__</code>	16
7.7	L195 関数 <code>TelemetryTextBridgeNode._set_outbound</code>	20
7.8	L198 関数 <code>TelemetryTextBridgeNode._poll_socket</code>	21
7.9	L238 関数 <code>TelemetryTextBridgeNode._publish_gps</code>	23
7.10	L253 関数 <code>TelemetryTextBridgeNode._handle_vehicle_payload</code>	24
7.11	L291 関数 <code>TelemetryTextBridgeNode._handle_chase_payload</code>	27
7.12	L305 関数 <code>TelemetryTextBridgeNode._publish_weather</code>	28
7.13	L317 関数 <code>TelemetryTextBridgeNode._send_payload</code>	29
7.14	L325 関数 <code>TelemetryTextBridgeNode._send_outbound</code>	30
7.15	L349 関数 <code>main</code>	31
7.16	設定値と入力パラメータ	32
7.17	ROS topic I/O	33
8	処理の流れ	34
9	このファイルの位置づけ	34

1 対象

項目	内容
ファイル	mpc_solarcar/telemetry_text_bridge_node.py
ソース SHA-256	8eb8c10611ec3d5115db88a38834178bead9b3c8ef68d952df090bc4fe2c6aa9
種別	Python
区分	runtime node
役割	車両側・伴走車側から届く UDP 文字列を ROS topic に変換し、planner 指令を逆向きに文字列送信する。
起動文脈	live_wifi のセンサ入口と outbound command bridge を兼ねる。

2 このファイルを読む理由

車両側・伴走車側から届く UDP 文字列を ROS topic に変換し、planner 指令を逆向きに文字列送信する。

- inbound と outbound の両方向を持つ。
- speed、battery、distance、GPS、wind を ROS topic 化する。
- planner command を JSON/テキストで送り返す。

3 呼び出し関係

3.1 どこから呼ばれるか

- launch/solar_race_live_wifi.launch.py

3.2 次に読むと理解が進むファイル

- mpc_solarcar/telemetry_protocol.py
- mpc_solarcar/speed_command_bridge_node.py

3.3 同一リポジトリ内の主要依存

- mpc_solarcar/signal_utils.py
- mpc_solarcar/telemetry_protocol.py

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
return float(wind_speed * math.cos(rel))

class TelemetryTextBridgeNode(Node):
```

```

def __init__(self) -> None:
    super().__init__("telemetry_text_bridge_node")
    self.declare_parameter("enable_inbound", True)
    self.declare_parameter("enable_outbound", True)
    self.declare_parameter("bind_host", "0.0.0.0")
    self.declare_parameter("bind_port", 52001)
    self.declare_parameter("publish_period_sec", 1.0)
    self.declare_parameter("solar_remote_host", "192.168.50.21")
    self.declare_parameter("solar_remote_port", 52002)
    self.declare_parameter("chase_remote_host", "192.168.50.22")
    self.declare_parameter("chase_remote_port", 52003)
    self.declare_parameter("send_to_solar", True)
    self.declare_parameter("send_to_chase", True)
    self.declare_parameter("prefer_direct_distance", False)
    self.declare_parameter("speed_filter_tau_sec", 0.6)
    self.declare_parameter("speed_max_kmh", 130.0)
    self.declare_parameter("speed_max_accel_kmhps", 12.0)
    self.declare_parameter("speed_max_decel_kmhps", 20.0)

```

5 主要構造の読み取り

主要クラスは TelemetryTextBridgeNode。主要関数は parse_text_payload, headwind_component_ms, main。ROS パラメータ宣言は 27 件。ROS I/O は publisher 20 件、subscription 6 件。

5.1 主要クラス・関数

行	種別	名前	補足
79	class	TelemetryTextBridgeNode	bases: Node
19	function	_safe_float	args: payload
32	function	_parse_key_value_text	args: text
53	function	parse_text_payload	args: text
66	function	headwind_component_ms	args: payload
80	function	__init__	args: self
195	function	_set_outbound	args: self, key, value
198	function	_poll_socket	args: self
238	function	_publish_gps	args: self, pub, lat, lon, alt, source_unix
253	function	_handle_vehicle_payload	args: self, payload, source_unix
291	function	_handle_chase_payload	args: self, payload, source_unix
305	function	_publish_weather	args: self, payload, wind_speed, wind_dir, course_deg, headwind, now
317	function	_send_payload	args: self, addr, payload
325	function	_send_outbound	args: self
349	function	main	

5.2 ファイルを上から読んだときの定義順

行	内容
19	関数 _safe_float を定義する。

32 関数 `_parse_key_value_text` を定義する。
53 関数 `parse_text_payload` を定義する。
66 関数 `headwind_component_ms` を定義する。
79 クラス `TelemetryTextBridgeNode` を定義する。
349 関数 `main` を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from __future_ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>import json</code>	manifest、checkpoint、UDP payload をやり取りするため。このファイル内での主な使用位置は L58, L321。
4	<code>import math</code>	Python 標準のスカラー数値関数と定数を使うため。実際に参照する名前と行は使用位置欄で確認する。このファイル内での主な使用位置は L27, L75, L76, L180, L181, L182。
5	<code>import socket</code>	socket モジュールを利用するため。このファイル内での主な使用位置は L166, L169。
6	<code>import time</code>	wall/monotonic time に基づく周期制御や freshness 判定を行うため。このファイル内での主な使用位置は L216, L254, L292, L330。
7	<code>from statistics import NormalDist</code>	statistics から NormalDist を読み込み、このファイルの処理を組み立てるため。このファイル内での使用位置は少ないか、間接利用である。
9	<code>import rclpy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L350, L352, L354。
10	<code>from rclpy.node import Node</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L79。
12	<code>from sensor_msgs. msg import NavSatFix</code>	GPS 緯度経度高度を ROS topic でやり取りするため。このファイル内での主な使用位置は L153, L154, L241。
13	<code>from std_msgs. msg import Float32, Float64, String</code>	Float32、Bool、String などの標準 ROS message で値を publish または subscribe するため。このファイル内での主な使用位置は L143, L144, L145, L146, L147, L148, L149, L150, ...。
15	<code>from .signal_ utils import RobustScalarFilter</code>	signal_utils.py から RobustScalarFilter を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/signal_utils.py</code> 。このファイル内での主な使用位置は L124, L132, L133, L134, L135, L136, L137。
16	<code>from .telemetry_ protocol import utc_iso_now, validate_source_timestamp</code>	telemetry_protocol.py から utc_iso_now, validate_source_timestamp を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/telemetry_protocol.py</code> 。このファイル内での主な使用位置は L214, L331。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1
```

```
counter = Counter()
counter.increment()
```

‘`super().__init__(...)`’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体が作られず、‘`create_publisher`’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、`__init__`、名前付けの作法

‘`self.step_solar`’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘`__init__`’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘`_`’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘`_base_csv`’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 rclpy、rcl、rmw、DDS/RTPS の層

rclpy は Python 利用者向け ROS 2 client library である。利用者の Node、publisher、subscriptionなどを Python API として提供し、その下で共通 C 層の rcl を利用する。

本プロジェクトのPythonコード -> rclpy -> rcl -> rmw -> DDS/RTPS実装 -> ネットワークまたは同一PC内通信

rcl は言語に依存しない共通 ROS 機能を提供する C API、rmw は ROS 2 と具体的 middleware 実装の境界である。DDS/RTPS 側が探索、serialize、publish/subscribe、request/replyなどを担う。executorの実行モデルは rcl だけで完結せず client library 側にも実装される。

根拠資料

- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.9 ROS graph、Node、topic、service、action、parameter

ROS graph は、実行中 Node と、それらが持つ publisher、subscription、service、actionなどの接続関係である。Python クラス、プロセス、ROS graph 上の Node 名は関連するが同一物ではない。

topic は継続データ向けの非同期一方向 publish/subscribe、service は短い request/response、action は時間のかかる目標へ feedback、cancel、result を持たせる。parameter は Node ごとの設定値である。

publisher が送った時点と subscription callback が実行される時点は同じとは限らない。通信遅延、QoS queue、executor 待ちを挟むため、センサ値には timestamp と freshness 判定が必要になる。

根拠資料

- [ROS 2 Humble 公式: Understanding nodes](#)
- [ROS 2 Humble 公式: Topics, Services, Actions](#)
- [ROS 2 Humble 公式: Parameters](#)

6.10 callback、timer、Executor、spin、Callback Group

callback は、メッセージ受信、timer 満了、service request などのイベントが成立した後で Executor から呼ばれる関数である。登録時に 'self._on_speed' と括弧なしで渡すのは、今実行せず後で呼ぶ関数オブジェクトを渡すためである。

Executor は実行可能になった callback を見つけ、Callback Group の条件を確認して実行する。'spin()' は終了要求まで待機・dispatch を続けるため、通常運転中は main の次の行へ戻らない。

MutuallyExclusiveCallbackGroup は同じ group 内の callback を同時実行させない。別 group 間は MultiThreadedExecutor で並行実行し得る。group を分けただけでは共有属性への完全な排他にはならないため、複数 group が同じ状態を読む・書く場合は設計確認が必要である。

```
DDS受信またはtimer満了 -> wait setでready -> Executorが選択 -> Callback Groupが許可 -> worker threadがcallback実行 -> 終了後Executorへ戻る
```

根拠資料

- [ROS 2 公式: Executors](#)
- [ROS 2 Humble 公式: Internal ROS 2 interfaces](#)

6.11 rqt_graph、ros2 CLI、rosbag2 をいつ使うか

rqt_graph は接続関係を見る道具であり、数値の正しさや更新周期までは保証しない。起動直後、topic 名変更後、publisher が複数存在する疑いがある時に使う。

```
ros2 node list
ros2 node info /mpc_node
ros2 topic list -t
ros2 topic info -v /planner/speed_cmd
ros2 topic hz /planner/speed_cmd
ros2 topic echo /planner/status
rqt_graph
```

rosbag2 は topic メッセージを時系列のまま記録・再生する。通信不具合、freshness、再計画 trigger、実車と SILS の差を再現可能にするため、本番前試験では制御入力だけでなく原因となる全 telemetry、status、parameter 情報を記録する。

```
ros2 bag record -o outputs/bags/preflight \
  /vehicle/s_km /vehicle/speed_kmh /vehicle/batt_soc \
  /vehicle/batt_temp_c /vehicle/batt_current_a /vehicle/batt_voltage_v \
  /planner/upper_speed_cmd /planner/speed_cmd /planner/status

ros2 bag info outputs/bags/preflight
ros2 bag play outputs/bags/preflight --clock
```

bag 再生時は QoS 互換性、simulation time、外部 publisher との二重入力に注意する。実車 Node を同時に動かす場合は namespace または remapping で入力源を明確に分離する。

根拠資料

- [ROS 2 Humble 公式: rqt_graph](#)
- [ROS 2 Humble 公式: Recording and playing back data](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.12 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ' $x[k+1] = f(x[k], u[k], w[k])$ ' と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- [SciPy 公式: scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L19 関数 `_safe_float`

- 行範囲: L19–L29
- 所属: module 直下
- 定義: `_safe_float(payload:dict,*keys:str)->float|None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `isfinite`
- 戻り値の要点: `None` / `value`
- 主なローカル代入名: `key`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 1、`try` 1

この定義を読むための Python 構文

- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. keys を順に走査し、各要素を key に入れて処理する。
2. 条件 `key not in payload` を判定し、真なら内部処理を行う。
3. `Continue` 文を実行する。
4. 例外処理を伴う `try` ブロックを実行する。
5. value に `float(payload[key])` の結果を代入する。
6. `Exception` を捕捉した場合:
7. `Continue` 文を実行する。
8. 条件 `math.isfinite(value)` を判定し、真なら内部処理を行う。
9. value を返す。
10. `None` を返す。

代表コード断片

```
def _safe_float(payload: dict, *keys: str) -> float | None:
    for key in keys:
        if key not in payload:
            continue
        try:
            value = float(payload[key])
        except Exception:
            continue
        if math.isfinite(value):
            return value
    return None
```

7.2 L32 関数 `_parse_key_value_text`

- 行範囲: L32–L50
- 所属: module 直下
- 定義: `_parse_key_value_text(text:str)->dict`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `replace`, `split`, `strip`
- 戻り値の要点: `payload`
- 主なローカル代入名: `key`, `payload`, `sep`, `token`, `value`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 1、`try` 1

この定義を読むための Python 構文

- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. payload に {} を代入する。
2. text.replace(';', ',').replace('\n', ',').split(',') を順に走査し、各要素を token に入れて処理する。
3. token に token.strip() の結果を代入する。
4. 条件 not token を判定し、真なら内部処理を行う。
5. Continue 文を実行する。
6. sep に '=' if '=' in token else ':' if ':' in token else '' の結果を代入する。
7. 条件 not sep を判定し、真なら内部処理を行う。
8. Continue 文を実行する。
9. (key, value) に token.split(sep, 1) の結果を代入する。
10. key に key.strip() の結果を代入する。
11. value に value.strip() の結果を代入する。
12. 条件 not key を判定し、真なら内部処理を行う。
13. Continue 文を実行する。
14. 例外処理を伴う try ブロックを実行する。
15. payload[key] に float(value) の結果を代入する。
16. Exception を捕捉した場合:
17. payload[key] に value の結果を代入する。
18. payload を返す。

代表コード断片

```
def _parse_key_value_text(text: str) -> dict:
    payload: dict[str, object] = {}
    for token in text.replace(";", ",").replace("\n", ",").split(","):
        token = token.strip()
        if not token:
            continue
        sep = "=" if "=" in token else ":" if ":" in token else ""
        if not sep:
            continue
        key, value = token.split(sep, 1)
        key = key.strip()
        value = value.strip()
        if not key:
            continue
        try:
            payload[key] = float(value)
        except Exception:
            payload[key] = value
    return payload
```

7.3 L53 関数 `parse_text_payload`

- 行範囲: L53–L63
- 所属: module 直下
- 定義: `parse_text_payload(text:str)->dict`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_parse_key_value_text`, `isinstance`, `loads`, `str`, `strip`
- 戻り値の要点: `_parse_key_value_text(raw) / {} / parsed`
- 主なローカル代入名: `parsed`, `raw`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 1

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. `raw` に `str(text or "").strip()` の結果を代入する。
2. 条件 `not raw` を判定し、真なら内部処理を行う。
3. `{}` を返す。
4. 例外処理を伴う try ブロックを実行する。
5. `parsed` に `json.loads(raw)` の結果を代入する。
6. 条件 `isinstance(parsed, dict)` を判定し、真なら内部処理を行う。
7. `parsed` を返す。
8. Exception を捕捉した場合:
9. Pass 文を実行する。
10. `_parse_key_value_text(raw)` を返す。

代表コード断片

```
def parse_text_payload(text: str) -> dict:
    raw = str(text or "").strip()
    if not raw:
        return {}
    try:
        parsed = json.loads(raw)
        if isinstance(parsed, dict):
            return parsed
    except Exception:
        pass
    return _parse_key_value_text(raw)
```

7.4 L66 関数 headwind_component_ms

- 行範囲: L66–L76
- 所属: module 直下
- 定義: headwind_component_ms(payload:dict)->float|None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: _safe_float, cos, float, radians
- 戻り値の要点: float(wind_speed * math.cos(rel)) / direct / None
- 主なローカル代入名: course_deg, direct, rel, wind_dir, wind_speed
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 2、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. direct に _safe_float(payload, 'headwind_ms') の結果を代入する。
2. 条件 direct is not None を判定し、真なら内部処理を行う。
3. direct を返す。
4. wind_speed に _safe_float(payload, 'wind_speed_ms', 'wind_ms') の結果を代入する。
5. wind_dir に _safe_float(payload, 'wind_dir_deg') の結果を代入する。
6. course_deg に _safe_float(payload, 'course_deg', 'heading_deg') の結果を代入する。
7. 条件 wind_speed is None or wind_dir is None or course_deg is None を判定し、真なら内部処理を行う。
8. None を返す。
9. rel に math.radians((wind_dir - course_deg) % 360.0) の結果を代入する。
10. float(wind_speed * math.cos(rel)) を返す。

代表コード断片

```
def headwind_component_ms(payload: dict) -> float | None:
    direct = _safe_float(payload, "headwind_ms")
    if direct is not None:
        return direct
    wind_speed = _safe_float(payload, "wind_speed_ms", "wind_ms")
    wind_dir = _safe_float(payload, "wind_dir_deg")
    course_deg = _safe_float(payload, "course_deg", "heading_deg")
    if wind_speed is None or wind_dir is None or course_deg is None:
        return None
    rel = math.radians((wind_dir - course_deg) % 360.0)
    return float(wind_speed * math.cos(rel))
```

7.5 L79 クラス TelemetryTextBridgeNode

- 行範囲: L79–L346
- 所属: module 直下
- 定義: `TelemetryTextBridgeNode(bases=Node)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 関数 `__init__` を定義する。
2. 関数 `_set_outbound` を定義する。
3. 関数 `_poll_socket` を定義する。
4. 関数 `_publish_gps` を定義する。
5. 関数 `_handle_vehicle_payload` を定義する。
6. 関数 `_handle_chase_payload` を定義する。
7. 関数 `_publish_weather` を定義する。
8. 関数 `_send_payload` を定義する。
9. 関数 `_send_outbound` を定義する。

代表コード断片

```
class TelemetryTextBridgeNode(Node):
    def __init__(self) -> None:
        super().__init__("telemetry_text_bridge_node")
        self.declare_parameter("enable_inbound", True)
        self.declare_parameter("enable_outbound", True)
        self.declare_parameter("bind_host", "0.0.0.0")
        self.declare_parameter("bind_port", 52001)
        self.declare_parameter("publish_period_sec", 1.0)
        self.declare_parameter("solar_remote_host", "192.168.50.21")
        self.declare_parameter("solar_remote_port", 52002)
        self.declare_parameter("chase_remote_host", "192.168.50.22")
        self.declare_parameter("chase_remote_port", 52003)
        self.declare_parameter("send_to_solar", True)
        self.declare_parameter("send_to_chase", True)
        self.declare_parameter("prefer_direct_distance", False)
        self.declare_parameter("speed_filter_tau_sec", 0.6)
        self.declare_parameter("speed_max_kmh", 130.0)
        self.declare_parameter("speed_max_accel_kmhps", 12.0)
        self.declare_parameter("speed_max_decel_kmhps", 20.0)
        self.declare_parameter("distance_max_rate_kmps", 0.06)
        self.declare_parameter("distance_max_backtrack_km", 0.02)
        self.declare_parameter("battery_filter_tau_sec", 1.0)
        self.declare_parameter("wind_filter_tau_sec", 1.0)
        self.declare_parameter("headwind_filter_tau_sec", 0.8)
        self.declare_parameter("max_abs_headwind_ms", 25.0)
        self.declare_parameter("timestamp_required", True)
        self.declare_parameter("max_packet_age_sec", 5.0)
        self.declare_parameter("max_future_skew_sec", 2.0)
        self.declare_parameter("max_out_of_order_sec", 0.0)
        self.declare_parameter("solar_power_gain_to_pack", 1.0)

        self.enable_inbound = bool(self.get_parameter("enable_inbound").value)
        self.enable_outbound = bool(self.get_parameter("enable_outbound").value)
        self.publish_period_sec = max(0.1, float(self.get_parameter("publish_period_sec").value))
        self.prefer_direct_distance = bool(self.get_parameter("prefer_direct_distance").value)
    ...
```

7.6 L80 関数 TelemetryTextBridgeNode.__init__

- 行範囲: L80–L193
- 所属: TelemetryTextBridgeNode
- 定義: __init__(self)->None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: RobustScalarFilter, __init__, _set_outbound, bind, bool, create_publisher, create_subscription, create_timer, declare_parameter, float, get_logger, get_parameter
- 戻り値の要点: 戻り値が無い、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: self.poll_socket, self.send_outbound, self.set_outbound, self.create_publisher, self.create_subscription, self.create_timer, self.declare_parameter, self.enable_inbound, self.enable_outbound, self.get_logger, self.get_parameter, self.max_abs_headwind_ms, self.publish_period_sec, self.sock

- 更新する主なインスタンス属性: `self.chase_remote`, `self.enable_inbound`, `self.enable_outbound`, `self.headwind_filter`, `self.i_filter`, `self.last_source_unix`, `self.max_abs_headwind_ms`, `self.max_future_skew_sec`, `self.max_out_of_order_sec`, `self.max_packet_age_sec`, `self.outbound_state`, `self.prefer_direct_distance`, `self.pub_chase_alt`, `self.pub_chase_gps`, `self.pub_chase_source_time`, `self.pub_course_dir`, `self.pub_headwind`, `self.pub_raw_chase`, `self.pub_raw_solar`, `self.pub_solar_source_time`, `self.pub_status`, `self.pub_vehicle_alt`, `self.pub_vehicle_gps`, `self.pub_vehicle_i`
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `lambda` は名前を付けずに短い関数オブジェクトを作る構文である。

上から順の処理

1. `super().__init__(...)` を実行する。
2. `self.declare_parameter(...)` を実行する。
3. `self.declare_parameter(...)` を実行する。
4. `self.declare_parameter(...)` を実行する。
5. `self.declare_parameter(...)` を実行する。
6. `self.declare_parameter(...)` を実行する。
7. `self.declare_parameter(...)` を実行する。
8. `self.declare_parameter(...)` を実行する。
9. `self.declare_parameter(...)` を実行する。
10. `self.declare_parameter(...)` を実行する。
11. `self.declare_parameter(...)` を実行する。
12. `self.declare_parameter(...)` を実行する。
13. `self.declare_parameter(...)` を実行する。
14. `self.declare_parameter(...)` を実行する。
15. `self.declare_parameter(...)` を実行する。
16. `self.declare_parameter(...)` を実行する。
17. `self.declare_parameter(...)` を実行する。
18. `self.declare_parameter(...)` を実行する。
19. `self.declare_parameter(...)` を実行する。
20. `self.declare_parameter(...)` を実行する。
21. `self.declare_parameter(...)` を実行する。

22. self.declare_parameter(...) を実行する。
23. self.declare_parameter(...) を実行する。
24. self.declare_parameter(...) を実行する。
25. self.declare_parameter(...) を実行する。
26. self.declare_parameter(...) を実行する。
27. self.declare_parameter(...) を実行する。
28. self.declare_parameter(...) を実行する。
29. self.enable_inbound に bool(self.get_parameter('enable_inbound').value) の結果を代入する。
30. self.enable_outbound に bool(self.get_parameter('enable_outbound').value) の結果を代入する。
31. self.publish_period_sec に max(0.1, float(self.get_parameter('publish_period_sec').value)) の結果を代入する。
32. self.prefer_direct_distance に bool(self.get_parameter('prefer_direct_distance').value) の結果を代入する。
33. self.max_abs_headwind_ms に max(0.0, float(self.get_parameter('max_abs_headwind_ms').value)) の結果を代入する。
34. self.timestamp_required に bool(self.get_parameter('timestamp_required').value) の結果を代入する。
35. self.max_packet_age_sec に max(0.0, float(self.get_parameter('max_packet_age_sec').value)) の結果を代入する。
36. self.max_future_skew_sec に max(0.0, float(self.get_parameter('max_future_skew_sec').value)) の結果を代入する。
37. self.max_out_of_order_sec に max(0.0, float(self.get_parameter('max_out_of_order_sec').value)) の結果を代入する。
38. self.solar_power_gain_to_pack に max(0.0, float(self.get_parameter('solar_power_gain_to_pack').value)) の結果を代入する。
39. self.last_source_unix に {'solar': None, 'chase': None} の結果を代入する。
40. self.speed_filter に RobustScalarFilter(tau_sec=float(self.get_parameter('speed_filter_tau_sec').value), min_value=0.0, max_value=float(self.get_parameter('speed_max_kmh').value), rise_rate=float(self.get_parameter('speed_rise_rate').value), fall_rate=float(self.get_parameter('speed_max_decel_kmhps').value), initial_value=0.0) の結果を代入する。
41. self.soc_filter に RobustScalarFilter(tau_sec=float(self.get_parameter('battery_filter_tau_sec').value), min_value=0.0, max_value=1.0) の結果を代入する。
42. self.tb_filter に RobustScalarFilter(tau_sec=float(self.get_parameter('battery_filter_tau_sec').value), min_value=-40.0, max_value=120.0) の結果を代入する。
43. self.i_filter に RobustScalarFilter(tau_sec=float(self.get_parameter('battery_filter_tau_sec').value)) の結果を代入する。
44. self.v_filter に RobustScalarFilter(tau_sec=float(self.get_parameter('battery_filter_tau_sec').value), min_value=0.0, max_value=1000.0) の結果を代入する。
45. self.wind_filter に RobustScalarFilter(tau_sec=float(self.get_parameter('wind_filter_tau_sec').value)) の結果を代入する。
46. self.headwind_filter に RobustScalarFilter(tau_sec=float(self.get_parameter('headwind_filter_tau_sec').value), min_value=-self.max_abs_headwind_ms, max_value=self.max_abs_headwind_ms) の結果を代入する。

47. self.pub_raw_solar に self.create_publisher(String, '/telemetry/raw_solar', 10) の結果を代入する。
48. self.pub_raw_chase に self.create_publisher(String, '/telemetry/raw_chase', 10) の結果を代入する。
49. self.pub_vehicle_speed に self.create_publisher(Float32, '/vehicle/speed_kmh', 10) の結果を代入する。
50. self.pub_vehicle_soc に self.create_publisher(Float32, '/vehicle/batt_soc', 10) の結果を代入する。
51. self.pub_vehicle_tb に self.create_publisher(Float32, '/vehicle/batt_temp_c', 10) の結果を代入する。
52. self.pub_vehicle_i に self.create_publisher(Float32, '/vehicle/batt_current_a', 10) の結果を代入する。
53. self.pub_vehicle_v に self.create_publisher(Float32, '/vehicle/batt_voltage_v', 10) の結果を代入する。
54. self.pub_vehicle_solar に self.create_publisher(Float32, '/vehicle/solar_power_w', 10) の結果を代入する。
55. self.pub_vehicle_alt に self.create_publisher(Float32, '/vehicle/altitude_m', 10) の結果を代入する。
56. self.pub_vehicle_s に self.create_publisher(Float32, '/vehicle/s_kmh', 10) の結果を代入する。
57. self.pub_vehicle_gps に self.create_publisher(NavSatFix, '/vehicle/gps', 10) の結果を代入する。
58. self.pub_chase_gps に self.create_publisher(NavSatFix, '/chase/gps', 10) の結果を代入する。
59. self.pub_chase_alt に self.create_publisher(Float32, '/chase/altitude_m', 10) の結果を代入する。
60. self.pub_headwind に self.create_publisher(Float32, '/weather/headwind_meas_ms', 10) の結果を代入する。
61. self.pub_wind_speed に self.create_publisher(Float32, '/weather/wind_speed_ms', 10) の結果を代入する。
62. self.pub_wind_dir に self.create_publisher(Float32, '/weather/wind_dir_deg', 10) の結果を代入する。
63. self.pub_course_dir に self.create_publisher(Float32, '/weather/course_deg', 10) の結果を代入する。
64. self.pub_status に self.create_publisher(String, '/telemetry/bridge_status', 10) の結果を代入する。
65. self.pub_solar_source_time に self.create_publisher(Float64, '/telemetry/solar_source_ts_unix', 10) の結果を代入する。
66. self.pub_chase_source_time に self.create_publisher(Float64, '/telemetry/chase_source_ts_unix', 10) の結果を代入する。
67. self.sock に None の結果を代入する。
68. 条件 self.enable_inbound を判定し、真なら内部処理を行う。
69. self.sock に socket.socket(socket.AF_INET, socket.SOCK_DGRAM) の結果を代入する。
70. self.sock.bind(...) を実行する。
71. self.sock.setblocking(...) を実行する。
72. self.tx_sock に socket.socket(socket.AF_INET, socket.SOCK_DGRAM) if self.enable_outbound else None の結果を代入する。
73. self.solar_remote に (str(self.get_parameter('solar_remote_host').value), int(self.get_parameter('solar_remote_port').value)) の結果を代入する。
74. self.chase_remote に (str(self.get_parameter('chase_remote_host').value), int(self.get_parameter('chase_remote_port').value)) の結果を代入する。
75. self.send_to_solar に bool(self.get_parameter('send_to_solar').value) の結果を代入する。
76. self.send_to_chase に bool(self.get_parameter('send_to_chase').value) の結果を代入する。

77. self.outbound_state に {'speed_cmd_kmh': 0.0, 'upper_speed_cmd_kmh': 0.0, 'drive_mode': 'stop', 'speed_meas_kmh': math.nan, 'soc': math.nan, 's_kmh': math.nan} の結果を代入する。
78. self.create_subscription(...) を実行する。
79. self.create_subscription(...) を実行する。
80. self.create_subscription(...) を実行する。

代表コード断片

```
def __init__(self) -> None:
    super().__init__("telemetry_text_bridge_node")
    self.declare_parameter("enable_inbound", True)
    self.declare_parameter("enable_outbound", True)
    self.declare_parameter("bind_host", "0.0.0.0")
    self.declare_parameter("bind_port", 52001)
    self.declare_parameter("publish_period_sec", 1.0)
    self.declare_parameter("solar_remote_host", "192.168.50.21")
    self.declare_parameter("solar_remote_port", 52002)
    self.declare_parameter("chase_remote_host", "192.168.50.22")
    self.declare_parameter("chase_remote_port", 52003)
    self.declare_parameter("send_to_solar", True)
    self.declare_parameter("send_to_chase", True)
    self.declare_parameter("prefer_direct_distance", False)
    self.declare_parameter("speed_filter_tau_sec", 0.6)
    self.declare_parameter("speed_max_kmh", 130.0)
    self.declare_parameter("speed_max_accel_kmhps", 12.0)
    self.declare_parameter("speed_max_decel_kmhps", 20.0)
    self.declare_parameter("distance_max_rate_kmps", 0.06)
    self.declare_parameter("distance_max_backtrack_kmh", 0.02)
    self.declare_parameter("battery_filter_tau_sec", 1.0)
    self.declare_parameter("wind_filter_tau_sec", 1.0)
    self.declare_parameter("headwind_filter_tau_sec", 0.8)
    self.declare_parameter("max_abs_headwind_ms", 25.0)
    self.declare_parameter("timestamp_required", True)
    self.declare_parameter("max_packet_age_sec", 5.0)
    self.declare_parameter("max_future_skew_sec", 2.0)
    self.declare_parameter("max_out_of_order_sec", 0.0)
    self.declare_parameter("solar_power_gain_to_pack", 1.0)

    self.enable_inbound = bool(self.get_parameter("enable_inbound").value)
    self.enable_outbound = bool(self.get_parameter("enable_outbound").value)
    self.publish_period_sec = max(0.1, float(self.get_parameter("publish_period_sec").value))
    self.prefer_direct_distance = bool(self.get_parameter("prefer_direct_distance").value)
    self.max_abs_headwind_ms = max(0.0, float(self.get_parameter("max_abs_headwind_ms").value))
    ...
```

7.7 L195 関数 TelemetryTextBridgeNode._set_outbound

- 行範囲: L195–L196
- 所属: TelemetryTextBridgeNode
- 定義: _set_outbound(self, key:str, value)->None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。

- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: `self.outbound_state`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `self.outbound_state[key]` に `value` の結果を代入する。

代表コード断片

```
def _set_outbound(self, key: str, value) -> None:
    self.outbound_state[key] = value
```

7.8 L198 関数 `TelemetryTextBridgeNode._poll_socket`

- 行範囲: L198–L236
- 所属: `TelemetryTextBridgeNode`
- 定義: `_poll_socket(self) -> None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float64`, `String`, `_handle_chase_payload`, `_handle_vehicle_payload`, `decode`, `get`, `lower`, `parse_text_payload`, `publish`, `recvfrom`, `str`, `time`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `_addr`, `data`, `payload`, `payload_type`, `pub`, `source`, `text`, `validation`
- 読み取る主なインスタンス属性: `self._handle_chase_payload`, `self._handle_vehicle_payload`, `self.last_source_unix`, `self.max_future_skew_sec`, `self.max_out_of_order_sec`, `self.max_packet_age_sec`, `self.pub_chase_source_time`, `self.pub_raw_chase`, `self.pub_raw_solar`, `self.pub_solar_source_time`, `self.pub_status`, `self.sock`, `self.timestamp_required`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 5、ループ 1、`try` 1

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- `try` 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `self.sock is None` を判定し、真なら内部処理を行う。
2. を返す。
3. 条件 `True` が成り立つ間くり返す。
4. 例外処理を伴う `try` ブロックを実行する。
5. `(data, _addr)` に `self.sock.recvfrom(4096)` の結果を代入する。
6. `BlockingIOError` を捕捉した場合:
7. `Break` 文を実行する。
8. `Exception` を捕捉した場合:
9. `Break` 文を実行する。
10. `text` に `data.decode('utf-8', errors='replace')` の結果を代入する。
11. `payload` に `parse_text_payload(text)` の結果を代入する。
12. 条件 `not payload` を判定し、真なら内部処理を行う。
13. `Continue` 文を実行する。
14. `payload_type` に `str(payload.get('type', '')).lower()` の結果を代入する。
15. `source` に `'chase' if payload_type == 'chase_state' else 'solar'` の結果を代入する。
16. `validation` に `validate_source_timestamp(payload, now_unix=time.time(), last_source_unix=self.last_source_unix[source], required=self.timestamp_required, max_age_sec=self.max_packet_age_sec, max_future_skew_sec=self.max_future_skew_sec, max_out_of_order_sec=self.max_out_of_order_sec)` の結果を代入する。
17. 条件 `not validation.accepted` を判定し、真なら内部処理を行う。
18. `self.pub_status.publish(...)` を実行する。
19. `Continue` 文を実行する。
20. 条件 `validation.source_unix is not None` を判定し、真なら内部処理を行う。
21. `self.last_source_unix[source]` に `validation.source_unix` の結果を代入する。
22. `pub` に `self.pub_chase_source_time if source == 'chase' else self.pub_solar_source_time` の結果を代入する。
23. `pub.publish(...)` を実行する。
24. 条件 `source == 'chase'` を判定し、真なら内部処理を行う。
25. `self.pub_raw_chase.publish(...)` を実行する。
26. `self._handle_chase_payload(...)` を実行する。
27. 上の条件が偽の場合:
28. `self.pub_raw_solar.publish(...)` を実行する。
29. `self._handle_vehicle_payload(...)` を実行する。

代表コード断片

```
def _poll_socket(self) -> None:
    if self.sock is None:
        return
    while True:
        try:
            data, _addr = self.sock.recvfrom(4096)
        except BlockingIOError:
            break
        except Exception:
            break
        text = data.decode("utf-8", errors="replace")
        payload = parse_text_payload(text)
        if not payload:
            continue
        payload_type = str(payload.get("type", "")).lower()
        source = "chase" if payload_type == "chase_state" else "solar"
        validation = validate_source_timestamp(
            payload,
            now_unix=time.time(),
            last_source_unix=self.last_source_unix[source],
            required=self.timestamp_required,
            max_age_sec=self.max_packet_age_sec,
            max_future_skew_sec=self.max_future_skew_sec,
            max_out_of_order_sec=self.max_out_of_order_sec,
        )
        if not validation.accepted:
            self.pub_status.publish(String(data=f"rejected source={source} reason={validation.reason}"))
            continue
        if validation.source_unix is not None:
            self.last_source_unix[source] = validation.source_unix
            pub = self.pub_chase_source_time if source == "chase" else self.
pub_solar_source_time
            pub.publish(Float64(data=validation.source_unix))

        if source == "chase":
            self.pub_raw_chase.publish(String(data=text))
    ...
```

7.9 L238 関数 TelemetryTextBridgeNode._publish_gps

- 行範囲: L238–L251
- 所属: TelemetryTextBridgeNode
- 定義: `_publish_gps(self, pub, lat:float|None, lon:float|None, alt:float|None, source_unix:float|None) ->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `NavSatFix`, `float`, `get_clock`, `int`, `max`, `now`, `publish`, `to_msg`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `msg`, `nanoseconds`
- 読み取る主なインスタンス属性: `self.get_clock`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 2、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `lat is None or lon is None` を判定し、真なら内部処理を行う。
2. を返す。
3. `msg` に `NavSatFix()` の結果を代入する。
4. 条件 `source_unix is None` を判定し、真なら内部処理を行う。
5. `msg.header.stamp` に `self.get_clock().now().to_msg()` の結果を代入する。
6. 上の条件が偽の場合:
7. `nanoseconds` に `max(0, int(source_unix * 1000000000.0))` の結果を代入する。
8. `msg.header.stamp.sec` に `nanoseconds // 1000000000` の結果を代入する。
9. `msg.header.stamp.nanosec` に `nanoseconds % 1000000000` の結果を代入する。
10. `msg.latitude` に `float(lat)` の結果を代入する。
11. `msg.longitude` に `float(lon)` の結果を代入する。
12. `msg.altitude` に `float(alt) if alt is not None else 0.0` の結果を代入する。
13. `pub.publish(...)` を実行する。

代表コード断片

```
def _publish_gps(self, pub, lat: float | None, lon: float | None, alt: float | None,
source_unix: float | None) -> None:
    if lat is None or lon is None:
        return
    msg = NavSatFix()
    if source_unix is None:
        msg.header.stamp = self.get_clock().now().to_msg()
    else:
        nanoseconds = max(0, int(source_unix * 1.0e9))
        msg.header.stamp.sec = nanoseconds // 1_000_000_000
        msg.header.stamp.nanosec = nanoseconds % 1_000_000_000
    msg.latitude = float(lat)
    msg.longitude = float(lon)
    msg.altitude = float(alt) if alt is not None else 0.0
    pub.publish(msg)
```

7.10 L253 関数 `TelemetryTextBridgeNode._handle_vehicle_payload`

- 行範囲: L253–L289
- 所属: `TelemetryTextBridgeNode`
- 定義: `_handle_vehicle_payload(self, payload: dict, source_unix: float | None) -> None`
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `_publish_gps`, `_publish_weather`, `_safe_float`, `float`, `headwind_component_ms`, `monotonic`, `publish`, `update`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `alt`, `course_deg`, `current`, `headwind`, `lat`, `lon`, `now`, `s_km`, `soc`, `solar`, `speed`, `tb`, `voltage`, `wind_dir`, `wind_speed`
- 読み取る主なインスタンス属性: `self._publish_gps`, `self._publish_weather`, `self.i_filter`, `self.prefer_direct_distance`, `self.pub_vehicle_alt`, `self.pub_vehicle_gps`, `self.pub_vehicle_i`, `self.pub_vehicle_s`, `self.pub_vehicle_soc`, `self.pub_vehicle_solar`, `self.pub_vehicle_speed`, `self.pub_vehicle_tb`, `self.pub_vehicle_v`, `self.soc_filter`, `self.solar_power_gain_to_pack`, `self.speed_filter`, `self.tb_filter`, `self.v_filter`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 8、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `now` に `time.monotonic()` の結果を代入する。
2. `speed` に `_safe_float(payload, 'speed_kmh', 'vehicle_speed_kmh')` の結果を代入する。
3. `soc` に `_safe_float(payload, 'soc', 'batt_soc')` の結果を代入する。
4. `tb` に `_safe_float(payload, 'batt_temp_c', 'tb_c')` の結果を代入する。
5. `current` に `_safe_float(payload, 'batt_current_a', 'current_a')` の結果を代入する。
6. `voltage` に `_safe_float(payload, 'batt_voltage_v', 'voltage_v')` の結果を代入する。
7. `solar` に `_safe_float(payload, 'solar_power_w', 'pv_w')` の結果を代入する。
8. `s_km` に `_safe_float(payload, 's_kmh', 'distance_kmh')` の結果を代入する。
9. `lat` に `_safe_float(payload, 'lat', 'latitude')` の結果を代入する。
10. `lon` に `_safe_float(payload, 'lon', 'longitude')` の結果を代入する。
11. `alt` に `_safe_float(payload, 'alt_m', 'altitude_m')` の結果を代入する。
12. `wind_speed` に `_safe_float(payload, 'wind_speed_ms', 'wind_ms')` の結果を代入する。
13. `wind_dir` に `_safe_float(payload, 'wind_dir_deg')` の結果を代入する。
14. `course_deg` に `_safe_float(payload, 'course_deg', 'heading_deg')` の結果を代入する。
15. `headwind` に `headwind_component_ms(payload)` の結果を代入する。
16. 条件 `speed is not None` を判定し、真なら内部処理を行う。
17. `self.pub_vehicle_speed.publish(...)` を実行する。
18. 条件 `soc is not None` を判定し、真なら内部処理を行う。

19. self.pub_vehicle_soc.publish(...) を実行する。
20. 条件 tb is not None を判定し、真なら内部処理を行う。
21. self.pub_vehicle_tb.publish(...) を実行する。
22. 条件 current is not None を判定し、真なら内部処理を行う。
23. self.pub_vehicle_i.publish(...) を実行する。
24. 条件 voltage is not None を判定し、真なら内部処理を行う。
25. self.pub_vehicle_v.publish(...) を実行する。
26. 条件 solar is not None を判定し、真なら内部処理を行う。
27. self.pub_vehicle_solar.publish(...) を実行する。
28. 条件 alt is not None を判定し、真なら内部処理を行う。
29. self.pub_vehicle_alt.publish(...) を実行する。
30. 条件 self.prefer_direct_distance and s_km is not None を判定し、真なら内部処理を行う。
31. self.pub_vehicle_s.publish(...) を実行する。
32. self._publish_gps(...) を実行する。
33. self._publish_weather(...) を実行する。

代表コード断片

```
def _handle_vehicle_payload(self, payload: dict, source_unix: float | None) -> None:
    now = time.monotonic()
    speed = _safe_float(payload, "speed_kmh", "vehicle_speed_kmh")
    soc = _safe_float(payload, "soc", "batt_soc")
    tb = _safe_float(payload, "batt_temp_c", "tb_c")
    current = _safe_float(payload, "batt_current_a", "current_a")
    voltage = _safe_float(payload, "batt_voltage_v", "voltage_v")
    solar = _safe_float(payload, "solar_power_w", "pv_w")
    s_km = _safe_float(payload, "s_km", "distance_km")
    lat = _safe_float(payload, "lat", "latitude")
    lon = _safe_float(payload, "lon", "longitude")
    alt = _safe_float(payload, "alt_m", "altitude_m")
    wind_speed = _safe_float(payload, "wind_speed_ms", "wind_ms")
    wind_dir = _safe_float(payload, "wind_dir_deg")
    course_deg = _safe_float(payload, "course_deg", "heading_deg")
    headwind = headwind_component_ms(payload)

    if speed is not None:
        self.pub_vehicle_speed.publish(Float32(data=float(self.speed_filter.update(speed, now=now))))
    if soc is not None:
        self.pub_vehicle_soc.publish(Float32(data=float(self.soc_filter.update(soc, now=now))))
    if tb is not None:
        self.pub_vehicle_tb.publish(Float32(data=float(self.tb_filter.update(tb, now=now))))
    if current is not None:
        self.pub_vehicle_i.publish(Float32(data=float(self.i_filter.update(current, now=now))))
    if voltage is not None:
        self.pub_vehicle_v.publish(Float32(data=float(self.v_filter.update(voltage, now=now))))
    if solar is not None:
```

```

        self.pub_vehicle_solar.publish(
            Float32(data=float(solar) * self.solar_power_gain_to_pack)
        )
    if alt is not None:
        self.pub_vehicle_alt.publish(Float32(data=float(alt)))
    if self.prefer_direct_distance and s_km is not None:
        self.pub_vehicle_s.publish(Float32(data=float(s_km)))
...

```

7.11 L291 関数 TelemetryTextBridgeNode._handle_chase_payload

- 行範囲: L291–L303
- 所属: TelemetryTextBridgeNode
- 定義: `_handle_chase_payload(self, payload:dict, source_unix:float|None)->None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `_publish_gps`, `_publish_weather`, `_safe_float`, `float`, `headwind_component_ms`, `monotonic`, `publish`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `alt`, `course_deg`, `headwind`, `lat`, `lon`, `now`, `wind_dir`, `wind_speed`
- 読み取る主なインスタンス属性: `self._publish_gps`, `self._publish_weather`, `self.pub_chase_alt`, `self.pub_chase_gps`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `now` に `time.monotonic()` の結果を代入する。
2. `lat` に `_safe_float(payload, 'lat', 'latitude')` の結果を代入する。
3. `lon` に `_safe_float(payload, 'lon', 'longitude')` の結果を代入する。
4. `alt` に `_safe_float(payload, 'alt_m', 'altitude_m')` の結果を代入する。
5. `wind_speed` に `_safe_float(payload, 'wind_speed_ms', 'wind_ms')` の結果を代入する。
6. `wind_dir` に `_safe_float(payload, 'wind_dir_deg')` の結果を代入する。
7. `course_deg` に `_safe_float(payload, 'course_deg', 'heading_deg')` の結果を代入する。
8. `headwind` に `headwind_component_ms(payload)` の結果を代入する。
9. `self._publish_gps(...)` を実行する。

10. 条件 `alt is not None` を判定し、真なら内部処理を行う。
11. `self.pub_chase_alt.publish(...)` を実行する。
12. `self._publish_weather(...)` を実行する。

代表コード断片

```
def _handle_chase_payload(self, payload: dict, source_unix: float | None) -> None:
    now = time.monotonic()
    lat = _safe_float(payload, "lat", "latitude")
    lon = _safe_float(payload, "lon", "longitude")
    alt = _safe_float(payload, "alt_m", "altitude_m")
    wind_speed = _safe_float(payload, "wind_speed_ms", "wind_ms")
    wind_dir = _safe_float(payload, "wind_dir_deg")
    course_deg = _safe_float(payload, "course_deg", "heading_deg")
    headwind = headwind_component_ms(payload)
    self._publish_gps(self.pub_chase_gps, lat, lon, alt, source_unix)
    if alt is not None:
        self.pub_chase_alt.publish(Float32(data=float(alt)))
    self._publish_weather(payload, wind_speed, wind_dir, course_deg, headwind, now)
```

7.12 L305 関数 `TelemetryTextBridgeNode._publish_weather`

- 行範囲: L305–L315
- 所属: `TelemetryTextBridgeNode`
- 定義: `_publish_weather(self, payload: dict, wind_speed, wind_dir, course_deg, headwind, now: float) -> None`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Float32`, `String`, `float`, `get`, `publish`, `update`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `filtered`
- 読み取る主なインスタンス属性: `self.headwind_filter`, `self.pub_course_dir`, `self.pub_headwind`, `self.pub_status`, `self.pub_wind_dir`, `self.pub_wind_speed`, `self.wind_filter`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 4、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `‘->’` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `wind_speed is not None` を判定し、真なら内部処理を行う。
2. `self.pub_wind_speed.publish(...)` を実行する。
3. 条件 `wind_dir is not None` を判定し、真なら内部処理を行う。

4. self.pub_wind_dir.publish(...) を実行する。
5. 条件 course_deg is not None を判定し、真なら内部処理を行う。
6. self.pub_course_dir.publish(...) を実行する。
7. 条件 headwind is not None を判定し、真なら内部処理を行う。
8. filtered に self.headwind_filter.update(headwind, now=now) の結果を代入する。
9. self.pub_headwind.publish(...) を実行する。
10. self.pub_status.publish(...) を実行する。

代表コード断片

```
def _publish_weather(self, payload: dict, wind_speed, wind_dir, course_deg, headwind,
now: float) -> None:
    if wind_speed is not None:
        self.pub_wind_speed.publish(Float32(data=float(self.wind_filter.update(
wind_speed, now=now))))
    if wind_dir is not None:
        self.pub_wind_dir.publish(Float32(data=float(wind_dir)))
    if course_deg is not None:
        self.pub_course_dir.publish(Float32(data=float(course_deg)))
    if headwind is not None:
        filtered = self.headwind_filter.update(headwind, now=now)
        self.pub_headwind.publish(Float32(data=float(filtered)))
        self.pub_status.publish(String(data=f"headwind_ms={filtered:.2f} type={payload.
get('type', 'vehicle_state')}"))
```

7.13 L317 関数 TelemetryTextBridgeNode._send_payload

- 行範囲: L317-L323
- 所属: TelemetryTextBridgeNode
- 定義: _send_payload(self, addr: tuple[str, int], payload: dict) -> None
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: dumps, encode, sendto
- 戻り値の要点: 戻り値が無いが、return 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: self.tx_sock
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 1、ループ 0、try 1

この定義を読むための Python 構文

- 第 1 引数 self は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- try 文は例外が起き得る処理と、異常時または終了時の経路を分ける。

上から順の処理

1. 条件 `self.tx_sock is None` を判定し、真なら内部処理を行う。
2. を返す。
3. 例外処理を伴う `try` ブロックを実行する。
4. `self.tx_sock.sendto(...)` を実行する。
5. `Exception` を捕捉した場合:
6. を返す。

代表コード断片

```
def _send_payload(self, addr: tuple[str, int], payload: dict) -> None:
    if self.tx_sock is None:
        return
    try:
        self.tx_sock.sendto(json.dumps(payload, ensure_ascii=False).encode("utf-8"),
        addr)
    except Exception:
        return
```

7.14 L325 関数 `TelemetryTextBridgeNode._send_outbound`

- 行範囲: L325–L346
- 所属: `TelemetryTextBridgeNode`
- 定義: `_send_outbound(self) -> None`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `_send_payload`, `time`, `utc_iso_now`
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: `payload`
- 読み取る主なインスタンス属性: `self._send_payload`, `self.chase_remote`, `self.enable_outbound`, `self.outbound_state`, `self.send_to_chase`, `self.send_to_solar`, `self.solar_remote`
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、`try` 0

この定義を読むための Python 構文

- 第 1 引数 `self` は、このメソッドを呼んだインスタンス自身を受け取る慣習名である。
- `'->'` 以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `not self.enable_outbound` を判定し、真なら内部処理を行う。
2. を返す。
3. `payload` に `{'type': 'planner_command', 'ts_unix': time.time(), 'timestamp_utc': utc_iso_now(), 'planner': {'speed_cmd_kmh': self.outbound_state['speed_cmd_kmh'], 'upper_speed_cmd_kmh': self.outbound_state['upper_speed_cmd_kmh'], 'drive_mode': self.outbound_state['drive_mode']}, 'vehicle': {'speed_kmh': self.outbound_state['speed_meas_kmh'], 'soc': self.outbound_state['soc'], 's_kmh': self.outbound_state['s_kmh']}}` の結果を代入する。
4. 条件 `self.send_to_solar` を判定し、真なら内部処理を行う。
5. `self._send_payload(...)` を実行する。
6. 条件 `self.send_to_chase` を判定し、真なら内部処理を行う。
7. `self._send_payload(...)` を実行する。

代表コード断片

```
def _send_outbound(self) -> None:
    if not self.enable_outbound:
        return
    payload = {
        "type": "planner_command",
        "ts_unix": time.time(),
        "timestamp_utc": utc_iso_now(),
        "planner": {
            "speed_cmd_kmh": self.outbound_state["speed_cmd_kmh"],
            "upper_speed_cmd_kmh": self.outbound_state["upper_speed_cmd_kmh"],
            "drive_mode": self.outbound_state["drive_mode"],
        },
        "vehicle": {
            "speed_kmh": self.outbound_state["speed_meas_kmh"],
            "soc": self.outbound_state["soc"],
            "s_kmh": self.outbound_state["s_kmh"],
        },
    }
    if self.send_to_solar:
        self._send_payload(self.solar_remote, payload)
    if self.send_to_chase:
        self._send_payload(self.chase_remote, payload)
```

7.15 L349 関数 main

- 行範囲: L349–L354
- 所属: module 直下
- 定義: `main()->None`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `TelemetryTextBridgeNode`, `destroy_node`, `init`, `shutdown`, `spin`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 主なローカル代入名: `node`
- 読み取る主なインスタンス属性: 特になし。

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. `rclpy.init(...)` を実行する。
2. `node` に `TelemetryTextBridgeNode()` の結果を代入する。
3. `rclpy.spin(...)` を実行する。
4. `node.destroy_node(...)` を実行する。
5. `rclpy.shutdown(...)` を実行する。

代表コード断片

```
def main() -> None:
    rclpy.init()
    node = TelemetryTextBridgeNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

7.16 設定値と入力パラメータ

行	名前	既定値・補足
82	<code>enable_inbound</code>	<code>True</code>
83	<code>enable_outbound</code>	<code>True</code>
84	<code>bind_host</code>	<code>0.0.0.0</code>
85	<code>bind_port</code>	<code>52001</code>
86	<code>publish_period_sec</code>	<code>1.0</code>
87	<code>solar_remote_host</code>	<code>192.168.50.21</code>
88	<code>solar_remote_port</code>	<code>52002</code>
89	<code>chase_remote_host</code>	<code>192.168.50.22</code>
90	<code>chase_remote_port</code>	<code>52003</code>
91	<code>send_to_solar</code>	<code>True</code>
92	<code>send_to_chase</code>	<code>True</code>
93	<code>prefer_direct_distance</code>	<code>False</code>
94	<code>speed_filter_tau_sec</code>	<code>0.6</code>
95	<code>speed_max_kmh</code>	<code>130.0</code>
96	<code>speed_max_accel_kmhps</code>	<code>12.0</code>
97	<code>speed_max_decel_kmhps</code>	<code>20.0</code>
98	<code>distance_max_rate_kmps</code>	<code>0.06</code>
99	<code>distance_max_backtrack_km</code>	<code>0.02</code>
100	<code>battery_filter_tau_sec</code>	<code>1.0</code>

101	wind_filter_tau_sec	1.0
102	headwind_filter_tau_sec	0.8
103	max_abs_headwind_ms	25.0
104	timestamp_required	True
105	max_packet_age_sec	5.0
106	max_future_skew_sec	2.0
107	max_out_of_order_sec	0.0
108	solar_power_gain_to_pack	1.0

7.17 ROS topic I/O

行	種別	topic	callback / 補足
143	publisher	/telemetry/raw_solar	publisher
144	publisher	/telemetry/raw_chase	publisher
145	publisher	/vehicle/speed_kmh	publisher
146	publisher	/vehicle/batt_soc	publisher
147	publisher	/vehicle/batt_temp_c	publisher
148	publisher	/vehicle/batt_current_a	publisher
149	publisher	/vehicle/batt_voltage_v	publisher
150	publisher	/vehicle/solar_power_w	publisher
151	publisher	/vehicle/altitude_m	publisher
152	publisher	/vehicle/s_kmh	publisher
153	publisher	/vehicle/gps	publisher
154	publisher	/chase/gps	publisher
155	publisher	/chase/altitude_m	publisher
156	publisher	/weather/headwind_meas_ms	publisher
157	publisher	/weather/wind_speed_ms	publisher
158	publisher	/weather/wind_dir_deg	publisher
159	publisher	/weather/course_deg	publisher
160	publisher	/telemetry/bridge_status	publisher
161	publisher	/telemetry/solar_source_ts_unix	publisher
162	publisher	/telemetry/chase_source_ts_unix	publisher
184	subscription	/planner/speed_cmd	lambda msg: self._set_outbound('speed_cmd_kmh', float(msg.data))
185	subscription	/planner/upper_speed_cmd	lambda msg: self._set_outbound('upper_speed_cmd_kmh', float(msg.data))
186	subscription	/planner/drive_mode	lambda msg: self._set_outbound('drive_mode', str(msg.data))
187	subscription	/vehicle/speed_kmh	lambda msg: self._set_outbound('speed_meas_kmh', float(msg.data))

188	subscription	/vehicle/batt_soc	lambda	msg:
			self._set_outbound('soc',	
			float(msg.data))	
189	subscription	/vehicle/s_km	lambda	msg:
			self._set_outbound('s_km',	
			float(msg.data))	

8 処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/telemetry_text_bridge_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。